Documento Correcciones Proyecto 2

Cambios a la gramática de Objectilang

Proyecto	Proyecto 2 – Gramática de Objectilang
Integrantes	Juan Manuel Forero Bohórquez (202514666) Fergye Zhamyra Ortiz Gómez (202426021)

Introducción

Este documento reúne los cambios que hicimos a la gramática de Objectilang al pasar del Proyecto 1 al Proyecto 2. Partimos de nuestra propia solución del Proyecto 1 y la corregimos a partir de dos fuentes: las observaciones que recibimos en la rúbrica de esa entrega y los problemas que identificamos nosotros al revisar la gramática y al implementarla y probarla.

Correcciones de la rúbrica del Proyecto 1

La siguiente tabla resume, por cada criterio de la rúbrica, la observación recibida y la corrección que aplicamos.

Criterio	Observación recibida	Corrección
No terminales	Nombres inconsistentes y algunos redundantes	Se unificaron los nombres (`ListPar`, `ListParm`, `ListParam` y `ParmEsp` pasaron a `TypeParams`/`TypeParam`; `Param` quedó definido). `Cond`, `Ciclo`, `Asignación` y `Print` se integraron como alternativas de `Statement`. Se eliminaron `Digito`, `Ent` y `Primario` (redundantes con `IntLit`, `FloatLit` y las demás literales).
Terminales	`\n` no es un terminal; usar otra notación para letras y dígitos; el espacio y “;” no hacen parte del lenguaje	Quitamos `\n` del alfabeto (los saltos de línea son solo espacio en blanco). Letras y dígitos los describimos con rangos: `[a-z-A-Z]` y `[0-9]`. Agregamos los terminales que faltaban: `print`, `println`, `++`, `${`, `//`, `/* */` y `;` (este último solo como sentencia vacía). El espacio y `;` ya no se listan como caracteres de `Cadena`: una cadena acepta cualquier carácter salvo `"`, `\`, `$` aislado y saltos de línea.

Criterio	Observación recibida	Corrección
		Retiramos `->` y `=>` s del alfabeto porque ninguna regla los usa.
Reglas	No todas seguían EBNF	Reescribimos todas las reglas en una sola notación (sintaxis de Rascal, equivalente a EBNF).
Reglas especiales	Definiciones de tokens incorrectas; `Char` debía incluir las mayúsculas	`Id` ahora acepta mayúsculas y minúsculas (antes solo a–z, aunque el alfabeto incluía A–Z) y excluye las palabras reservadas. `Float` y `Ent` ya no incluyen el signo: el `-` se trata como operador unario. `Reescribimos Cadena` con escapes e interpolación bien definidos.

Errores de la gramática del Proyecto 1 que corregimos

Estos son los problemas que identificamos en la gramática del Proyecto 1 y el cambio correspondiente en el Proyecto 2.

Problema en el P1	Cambio en el P2
`Programa` incluía `methodDef`/`objectDef` (minúscula) y se definía con `(Programa)*` recursivo	`Program = Statement*`; nombres consistentes.
`Programa` no incluía `Print` ni expresiones, así que un método no podía devolver un valor ni llamar a otro	`Statement` incluye `print`, `println` y expresiones como sentencia.
`ObjectDef` no tenía cuerpo	Se agregó el cuerpo opcional `{ Statement* }`. Se mantuvo el orden del P1: `extends`, parámetros de tipo, parámetros.
`Primario` y `Invocación` aceptaban ambos un `id` simple (ambigüedad)	Se quitó `id` de las literales; un identificador es una `Invocation`.
`Invocación` exigía al menos un argumento: `f()` era imposible	Argumentos con `{Expression ","}*` (pueden ser cero).
`Asignación` usaba `Invocación` como nombre declarado (`val a.b.c = ...`)	Ahora es `("val" | "var") Id TypeAnn? "=" Expression`.
`Tipo` incluía los `:` también dentro de los argumentos de tipo (`[:Int, :Str]`)	Se separó `TypeAnn` (`":" TypeExpr`) de `TypeExpr`.

Problema en el P1	Cambio en el P2
new aceptaba argumentos sin paréntesis y un único MethodDef en el cuerpo	new Id TypeArgs? CallArgs? ObjectBody?, con paréntesis y MethodDef*. Los [...] de new e invocaciones son argumentos de tipo, sin +/-.
MethodDef usaba Param indefinido y paréntesis con un solo parámetro	Se usa Params (lista separada por comas, puede ser vacía).
Ciclo usaba int (no definido) y ++ (no estaba en el alfabeto); el for con tupla exigía siempre paréntesis	++ entra al alfabeto; la forma for (x <- xs) acepta un id o una tupla (a, b); el valor inicial del for por rango es una Expression.
Cond y Ciclo usaban Programa como cuerpo (ambiguo: no se sabe dónde termina)	El cuerpo es un bloque { ... } o una única sentencia (Body).
Print solo imprimía cadenas	Acepta cualquier expresión (las cadenas siguen siendo expresiones).
Expresiones ambiguas, sin precedencia ni asociatividad	Precedencias explícitas, de mayor a menor: literales, - y neg unarios, ** (derecha), * / %, + -, comparaciones (no asociativas), and, or.
Cadena permitía ${ Programa }	La interpolación es ${ Expression }.

Sintaxis concreta y resaltado

- Implementada con syntax/lexical de Rascal en Syntax.rsc, con espacios, // y /* */ como layout.
- Resaltado con @category: Comment, Constant (números y cadenas), Variable (identificadores) y Type (nombres de tipo). Las palabras reservadas se resaltan como keywords.
- Plugin.rsc registra la extensión *.obj en VS Code.

Sintaxis abstracta (AST)

- AST.rsc define los ADT AProgram, AStatement, AExpression, etc. Los constructores coinciden con las etiquetas de Syntax.rsc, y Implode.rsc usa implode para obtener el AST.
- Las partes opcionales (X?) se representan como list[X] con 0 o 1 elemento.
- Los paréntesis no generan nodo en el AST.

Ajustes durante la integración y las pruebas

Después de la primera versión, al probar el código con examples/ejemplo1.obj (que incluye el ejemplo del enunciado) se hicieron estos cambios adicionales a Syntax.rsc y AST.rsc:

- **Parámetros con val.** Param acepta también valParam: "val" Id ":" TypeExpr, necesario para object List [+A](val item : A). El AST tiene el constructor valParam en AParam para conservar el modificador.

- **Métodos sin signo igual.** `methodDef` tiene una segunda producción sin `=` antes del bloque, para aceptar `method main { ... }`. `overrideDef` conserva su regla con `=`.
- **Tipo de retorno opcional en la asignación de métodos.** En `methodAssign` (`c.inc = method(...) ... { }`) el tipo ahora es `TypeAnn?`. En el AST `returnType` pasó a `list[ATypeAnn]` y el campo de parámetros se llama `requiredParameters`.
- **Restricción de paréntesis en invocaciones y en new.** `Invocation` y `newObject` llevan `!>> "("`: un paréntesis que sigue a una invocación o a un `new` se toma siempre como lista de argumentos, y no como el inicio de otra sentencia `(expr)`. Esto evita la ambigüedad entre `f (x)` como una llamada o como dos sentencias.
- **Tokens auxiliares.** `Digits` es compartido por `IntLit` y `FloatLit` (la definición anterior de `FloatLit` era ambigua para Rascal). `TypeName` es el identificador de tipos, aparte de `Id`, para poder resaltarlo con la categoría `Type`.
- **Detalles de notación en Rascal.** Los operadores `<`, `>`, `<=`, `>=`, `<>` y la flecha `<-` se escriben escapados en el código (`"\<"`, etc.). La producción de paréntesis se etiqueta `bracket` para que no genere nodo en el AST.
- **Pruebas.** `Tests.rsc` comprueba la estructura del AST con patrones (sin comparar las anotaciones de ubicación) e incluye casos para parámetros `val`, métodos sin `=`, métodos asignados sin tipo, negativos, precedencia y asociatividad.

Decisiones y limitaciones a tener en cuenta

Estas son las decisiones de diseño que tomamos y las limitaciones que no resolvimos en la gramática.

- **No hay reasignación.** La gramática del P1 solo tiene `=` como igualdad dentro de expresiones, y `val`/`var` como declaraciones. Agregar `x = e` como sentencia la haría ambigua con la igualdad.
- **Sentencias sin separador.** Como las sentencias no llevan separador, una expresión que empieza con `-` (o con `(`) justo después de otra que termina en una expresión puede leerse de dos formas. Por ejemplo, `val x = 1 - 2` es ambiguo: puede ser la resta `1 - 2` o la sentencia `val x = 1` seguida de la sentencia `-2`.
- Las palabras clave `and`, `or` y `neg` se comportan como operadores; los identificadores no pueden usar las palabras reservadas.